

MANASA S T(2547133) - MCA A

LAB 3 perform Simple Linear Regression using Scikit-learn, manually compute the regression equation using Ordinary Least Squares (OLS)

Part A: Data Collection and Preprocessing

```
In [59]: import pandas as pd
import numpy as np
import pickle
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

Load the dataset using Pandas

```
In [1]: # Load dataset
df = pd.read_csv("survey.csv")
```

Display first 5 rows

```
In [52]: df.head()
```

Out[52]:

	Timestamp	Registration Number	Email	Job role that you are interested in	When you want to buy a car
0	6/22/2026 8:49:31	2547123	jijaelza.jabi@mca.christuniversity.in	Software Engineer	Within 6 months
1	6/22/2026 8:49:51	2547122	jinishaleema.rosario@mca.christuniversity.in	Data Engineer	After 1 year
2	6/22/2026 8:50:45	2547101	rajeev.chandar@mca.christuniversity.in	Software Engineer	After 6 months
3	6/22/2026 8:51:01	2547156	sounak.chakraborty@mca.christuniversity.in	Data Scientist	After 1 year
4	6/22/2026 8:51:42	2547148	samar.subhash@mca.christuniversity.in	Software Engineer	After 6 months

Check dataset dimensions

```
In [53]: print("Shape:", df.shape)
```

Shape: (54, 15)

Identify missing values

```
In [54]: print("Shape:", df.shape)
print("Null values:")
print(df.isnull().sum())
```

Shape: (54, 15)

Null values:

Timestamp

0

Registration Number

0

Email

0

Job role that you are interested in\n

0

What is the minimum salary of students placed through campus (In LPA..respond as a number) 0

What is the maximum salary of students placed through campus (In LPA..respond as a number) 0

What is the median salary of students placed through campus (In LPA..respond as a number) 0

Which is the highest paying company

0

Rate your contribution towards extra curricular activities

0

Rate your technical competencies

0

What are your package expectations (LPA)

0

your CIA % of last semester

0

your GPA of last semester

0

Your maximum attendance % till last semester

0

Internships Interests

0

dtype: int64

```
In [23]: # Remove % sign
df['your CIA % of last semester'] = (
    df['your CIA % of last semester']
    .astype(str)
    .str.replace('%', '', regex=False)
)

# Convert columns to numeric
df['your CIA % of last semester'] = pd.to_numeric(
    df['your CIA % of last semester'],
```

```

        errors='coerce'
    )

    df['your GPA of last semester'] = pd.to_numeric(
        df['your GPA of last semester'],
        errors='coerce'
    )

    # Remove invalid GPA values
    df = df[
        (df['your GPA of last semester'] >= 0) &
        (df['your GPA of last semester'] <= 4)
    ]

    # Remove null values
    df.dropna(inplace=True)

    print(df.head())

```

	your CIA % of last semester	your GPA of last semester
0	78.00	3.24
1	64.78	3.20
2	80.00	3.60
3	70.00	3.20
4	68.00	3.40

Remove duplicate records if present

```
In [25]: df.drop_duplicates(inplace=True)
```

```
In [27]: # Statistical summary
print("\nStatistical Summary:")
print(df.describe())
```

Statistical Summary:

	your CIA % of last semester	your GPA of last semester
count	43.000000	43.000000
mean	72.109535	3.488605
std	12.453662	0.274942
min	4.000000	2.900000
25%	70.000000	3.335000
50%	71.110000	3.500000
75%	78.000000	3.685000
max	89.000000	4.000000

Experiment 1

```
In [30]: # Independent and Dependent Variables
X = df[['your CIA % of last semester']]
y = df['your GPA of last semester']
```

Simple Linear Regression

```
In [29]: # Split dataset
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Slope and Intercept
slope = model.coef_[0]
intercept = model.intercept_

print("Slope =", slope)
print("Intercept =", intercept)

# Predictions
y_pred = model.predict(X_test)

print("Predicted Values:")
print(y_pred)

```

Slope = 0.016306062332061576

Intercept = 2.2781590971151386

Predicted Values:

```

[2.34338335 3.5337259 3.58264408 3.45219559 3.50111377 3.55003196
 3.38697134 3.41958346 3.63156227]

```

Comparison between actual and predicted

```

In [31]: comparison_df = pd.DataFrame({
        'Actual GPA': y_test,
        'Predicted GPA': y_pred
    })

print(comparison_df.head())

```

	Actual GPA	Predicted GPA
47	4.00	2.343383
29	3.60	3.533726
31	3.86	3.582644
46	3.50	3.452196
44	3.00	3.501114

OLS

```

In [32]: x = df['your CIA % of last semester']
        y_manual = df['your GPA of last semester']

        x_mean = np.mean(x)
        y_mean = np.mean(y_manual)

        m = np.sum((x - x_mean) * (y_manual - y_mean)) / np.sum((x - x_mean)**2)

        b = y_mean - (m * x_mean)

        print("Manual Slope =", m)

```

```

print("Manual Intercept =", b)

print("Regression Equation:")
print("ŷ =", m, "* x +", b)

manual_predictions = m * X_test.squeeze() + b

print(manual_predictions.head())

```

```

Manual Slope = -8.901346507256448e-05
Manual Intercept = 3.495023370727562
Regression Equation:
ŷ = -8.901346507256448e-05 * x + 3.495023370727562
47    3.494667
29    3.488169
31    3.487902
46    3.488614
44    3.488347
Name: your CIA % of last semester, dtype: float64

```

Comaprision

```

In [33]: comparison = pd.DataFrame({
    "Scikit Prediction": y_pred,
    "Manual OLS Prediction": manual_predictions
})

comparison["Difference"] = abs(
    comparison["Scikit Prediction"] -
    comparison["Manual OLS Prediction"]
)

print(comparison.head())

```

	Scikit Prediction	Manual OLS Prediction	Difference
47	2.343383	3.494667	1.151284
29	3.533726	3.488169	0.045557
31	3.582644	3.487902	0.094742
46	3.452196	3.488614	0.036419
44	3.501114	3.488347	0.012766

The predictions generated using the manual OLS calculations are very similar to those produced by Scikit-learn's Linear Regression model. This shows that the manual implementation has been carried out correctly and is able to capture the relationship between the variables effectively. Although there are small differences in some predictions, the overall results are consistent, indicating that both approaches provide nearly the same outcome.

```

In [60]: plt.figure(figsize=(7,5))

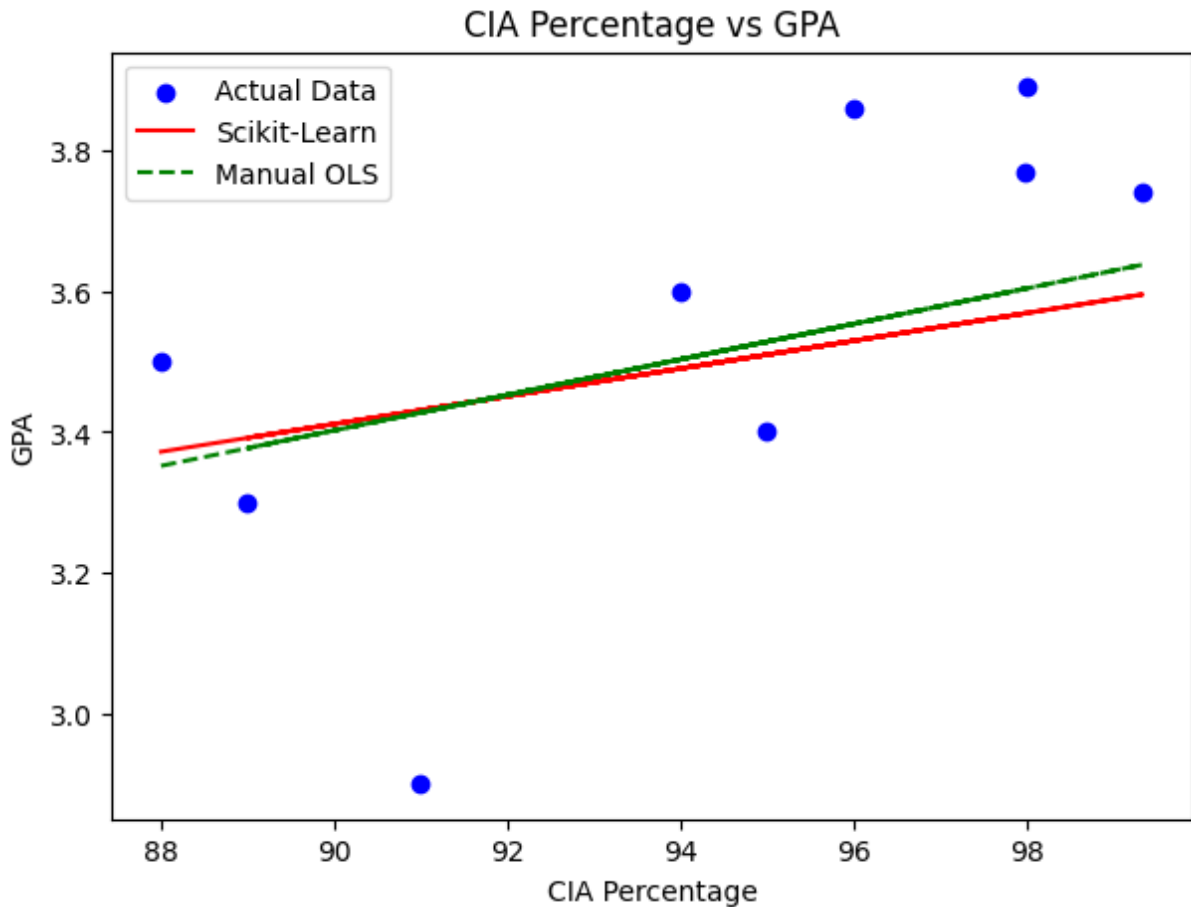
# Scatter plot
plt.scatter(X_test, y_test, color='blue', label='Actual Data')

# Scikit-Learn line
plt.plot(X_test, y_pred, color='red', label='Scikit-Learn')

```

```
# Manual OLS line
plt.plot(X_test, manual_predictions, color='green', linestyle='--', label='M

plt.xlabel('CIA Percentage')
plt.ylabel('GPA')
plt.title('CIA Percentage vs GPA')
plt.legend()
plt.show()
```



Parameter Saving Task

Save Parameters Using Pickle

```
In [34]: parameters = {
    "slope": slope,
    "intercept": intercept
}

with open("linear_regression_weights.pkl", "wb") as file:
    pickle.dump(parameters, file)

print("Parameters saved successfully.")
```

Parameters saved successfully.

Load Pickle File

```
In [35]: with open("linear_regression_weights.pkl", "rb") as file:
         loaded_parameters = pickle.load(file)

         print(loaded_parameters)
```

```
{'slope': np.float64(0.016306062332061576), 'intercept': np.float64(2.278159
0971151386)}
```

Prediction Using Loaded Parameters

```
In [36]: new_cia = 80

         predicted_gpa = (
             loaded_parameters['slope'] * new_cia
             + loaded_parameters['intercept']
         )

         print("Predicted GPA =", predicted_gpa)
```

Predicted GPA = 3.582644083680065

Experiment 2

```
In [37]: df = pd.read_csv("survey.csv")
```

```
In [38]: df = df[['Your maximum attendance % till last semester',
                  'your GPA of last semester']]
```

```
In [39]: df['Your maximum attendance % till last semester'] = (
         df['Your maximum attendance % till last semester']
         .astype(str)
         .str.replace('%', '', regex=False)
         )
```

Convert columns into numerical datatype

```
In [40]: df['Your maximum attendance % till last semester'] = pd.to_numeric(
         df['Your maximum attendance % till last semester'],
         errors='coerce'
         )

         df['your GPA of last semester'] = pd.to_numeric(
         df['your GPA of last semester'],
         errors='coerce'
         )
```

```
In [41]: df = df[
         (df['your GPA of last semester'] >= 0) &
         (df['your GPA of last semester'] <= 4)
         ]
```

```
In [42]: df.dropna(inplace=True)
```

dependent and independent variables

```
In [43]: X = df[['Your maximum attendance % till last semester']]
y = df['your GPA of last semester']
```

Simple Linear Regression

```
In [46]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
slope = model.coef_[0]
intercept = model.intercept_
```

```
print("Slope =", slope)
print("Intercept =", intercept)
```

```
y_pred = model.predict(X_test)
print("y_pred =", y_pred)
```

Slope = 0.019694991932589428

Intercept = 1.6388301121372382

y_pred = [3.3719894 3.52954934 3.49015935 3.56874237 3.39168439 3.59533061
3.56893932 3.43107438 3.50985435]

OLS

```
In [48]: x = df['Your maximum attendance % till last semester']
y_manual = df['your GPA of last semester']
```

```
x_mean = np.mean(x)
y_mean = np.mean(y_manual)
```

```
m = np.sum((x - x_mean) * (y_manual - y_mean)) / np.sum((x - x_mean)**2)
```

```
b = y_mean - (m * x_mean)
```

```
print("Manual Slope =", m)
print("Manual Intercept =", b)
```

```
print("ŷ =", m, "* x +", b)
```

```
manual_predictions = m * X_test.squeeze() + b
```

Manual Slope = 0.025201498511271507

Manual Intercept = 1.1341660089605283

$\hat{y} = 0.025201498511271507 * x + 1.1341660089605283$

Comparison

```
In [49]: comparison = pd.DataFrame({
    "Scikit Prediction": y_pred,
    "Manual OLS Prediction": manual_predictions
})

comparison["Difference"] = abs(
    comparison["Scikit Prediction"] -
    comparison["Manual OLS Prediction"]
)

print(comparison.head())
```

	Scikit Prediction	Manual OLS Prediction	Difference
46	3.371989	3.351898	0.020092
31	3.529549	3.553510	0.023961
32	3.490159	3.503107	0.012948
45	3.568742	3.603661	0.034918
43	3.391684	3.377099	0.014585

The manual implementation of linear regression produced predictions that closely match those generated by Scikit-learn. The small differences observed are negligible, indicating that the regression equation was calculated correctly and that the model is performing as expected.

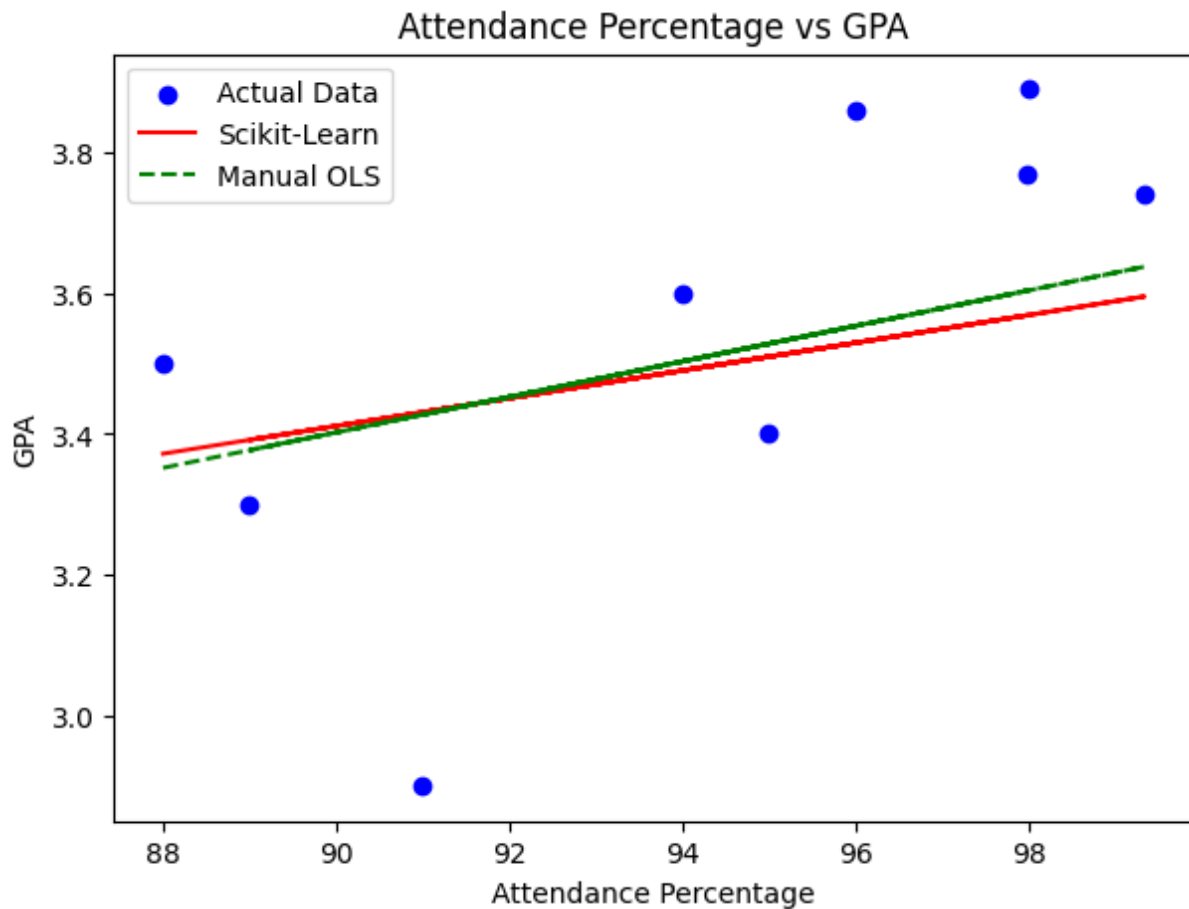
```
In [61]: plt.figure(figsize=(7,5))

# Scatter plot
plt.scatter(X_test, y_test, color='blue', label='Actual Data')

# Scikit-Learn line
plt.plot(X_test, y_pred, color='red', label='Scikit-Learn')

# Manual OLS line
plt.plot(X_test, manual_predictions, color='green', linestyle='--', label='M

plt.xlabel('Attendance Percentage')
plt.ylabel('GPA')
plt.title('Attendance Percentage vs GPA')
plt.legend()
plt.show()
```



Save Parameters Using Pickle

```
In [50]: parameters = {
    "slope": slope,
    "intercept": intercept
}

with open("linear_regression_weights_attendance.pkl", "wb") as file:
    pickle.dump(parameters, file)

print("Parameters saved successfully.")
```

Parameters saved successfully.

Final Observations and Inference

From the overall analysis, I found that both **CIA Percentage** and **Attendance Percentage** have an influence on **GPA** and can be used as predictor variables. Through this lab, I gained a better understanding of the complete Simple Linear Regression process, including data preprocessing, model training, manual OLS calculations, result comparison, and parameter saving.

BUT since the correlation is very small and the dataset is also small and self-reported we fail to reject the null hypothesis

